

به نام خدا

تمرین سری ۲ مدیریت امنیت اطلاعات

امیرحسین قدیری ۴۰۴۲۰۵۰۳۱

بخش ۲

سوال ۱

سیستم عامل مورد استفاده Arch Linux 64-bit می باشد. در ادامه مکانیزم های امنیتی موجود در این سیستم بررسی شده اند.

Secure Boot - ۱

Secure Boot بخشی از UEFI است که فقط اجازه اجرای Bootloader ها و Kernel هایی را می دهد که با کلیدهای مورد اعتماد امضا شده باشند. در نتیجه بدافزارهای سطح بوت مانند Bootkit و Rootkit نمی توانند قبل از سیستم عامل اجرا شوند. همانطور که در تصویر زیر مشاهده می شود، این سیستم از Secure Boot می کند ولی از آن استفاده نمی شود.

```
skye@arch ~> bootctl status
systemd-boot not installed in ESP.
No default/fallback boot loader installed in ESP.
System:
  Firmware: n/a (n/a)
  Firmware Arch: x64
  Secure Boot: disabled (setup)
  TPM2 Support: yes
  Measured UKI: no
  Boot into FW: supported
```

نقاط قوت Secure Boot به صورت زیر می باشند:

- جلوگیری از دستکاری فرآیند بوت
- جلوگیری از بسیاری از Bootkit ها
- ایجاد زنجیره اعتماد
- مناسب برای سیستم های سازمانی

نقاط ضعف Secure Boot به صورت زیر می باشند:

- پیکربندی آن در لینوکس نسبتاً پیچیده است.
- برخی توزیع ها نیاز به Shim و کلیدهای امضا شده دارند.
- در صورت مدیریت نامناسب کلیدها ممکن است امنیت کاهش یابد.
- از حملات پس از بالا آمدن سیستم جلوگیری نمی کند.

TPM - ۲

TPM یک ریشه اعتماد سخت افزاری است که کلیدهای رمزنگاری، اطلاعات هویتی و اندازه گیری های امنیتی بوت را ذخیره می کند. همانطور که در تصویر زیر مشاهده می شود، این سیستم از TPM 2.0 پشتیبانی می کند.

```
skye@arch ~> systemd-analyze has-tpm2
yes
+firmware
+driver
+system
+subsystem
+libraries
  +libtss2-esys.so.0
  +libtss2-rc.so.0
  +libtss2-mu.so.0
skye@arch ~> █
```

نقاط قوت TPM به صورت زیر می باشند:

- ذخیره امن کلیدهای رمزنگاری
- مناسب برای Full Disk Encryption
- مقاومت در برابر سرقت کلیدها
- پشتیبانی از Measured Boot

نقاط ضعف TPM به صورت زیر می باشند:

- TPM های Firmware در برخی تحقیقات آسیب پذیری هایی داشته اند.
- در صورت آسیب پذیری Firmware امکان دور زدن برخی حفاظت ها وجود دارد.
- برای کاربر عادی پیچیدگی مدیریتی دارد.

LUKS - ۳

LUKS مربوط به رمزنگاری دیسک بوده و از داده ها در صورت سرقت لپ تاپ یا هارد محافظت می کند. همانطور که در تصویر زیر مشاهده می شود، این سیستم از رمزنگاری دیسک استفاده نمی کند.

```

skye@arch ~> lsblk -f
NAME                FSTYPE FSVER LABEL UUID                                 FSAVAIL FSUSE% MOUNTPOINTS
nvme0n1
├─nvme0n1p1 vfat    FAT32    71FB-06C3                                767.6M    25% /boot
├─nvme0n1p2 swap    1        e511f67a-c981-4c74-8624-2cef188aca4d    [SWAP]
└─nvme0n1p3 btrfs   9044a6cc-4766-461b-84f2-d3cc74d73723    734G     20% /
skye@arch ~> 

```

نقاط قوت LUKS به صورت زیر می باشند:

- حفاظت قوی از داده های ذخیره شده
- استاندارد رایج لینوکس
- پشتیبانی از TPM

نقاط ضعف LUKS به صورت زیر می باشند:

- در زمان روشن بودن سیستم داده ها رمزگشایی شده اند.
- حملات Cold Boot همچنان ممکن هستند.
- فراموش کردن کلید برابر با از دست رفتن داده ها است.

۴- ASLR

با فعال بودن این مکانیزم، تصادفی سازی محل قرارگیری حافظه برای دشوار کردن بهره برداری از آسیب پذیری ها فعال می شود. با توجه به تصویر زیر، این مکانیزم در این سیستم فعال می باشد.

```

skye@arch ~> cat /proc/sys/kernel/randomize_va_space
2
skye@arch ~> 

```

نقاط قوت ASLR به صورت زیر می باشند:

- سخت تر شدن Exploit ها
- افزایش مقاومت در برابر Buffer Overflow

نقاط ضعف ASLR به صورت زیر می باشند:

- در صورت وجود Information Leak قابل دور زدن است.
- به تنهایی کافی نیست.

۵- NX Bit

فعال بودن NX Bit از اجرای کد در بخش هایی از حافظه مثل Stack که فقط مخصوص داده ها هستند جلوگیری می کند. با توجه به تصویر زیر، NX Bit در این سیستم فعال می باشد.

```
skye@arch ~> sudo dmesg | grep "NX "
[ 0.000000] NX (Execute Disable) protection: active
skye@arch ~> █
```

نقاط قوت NX Bit به صورت زیر می باشند:

- جلوگیری از Shellcode های کلاسیک
- کاهش موفقیت حملات Buffer Overflow

نقاط ضعف NX Bit به صورت زیر می باشند:

- در برابر ROP کافی نیست.
- در برابر JOP کافی نیست.

۶- Kernel Lockdown

حذف این مکانیزم محدود کردن دسترسی root به بخش های حساس Kernel می باشد. با توجه به تصویر زیر، این مکانیزم در سیستم فعلی غیرفعال نمی کند.

```
skye@arch ~> cat /sys/kernel/security/lockdown
[none] integrity confidentiality
skye@arch ~> █
```

نقاط قوت Kernel Lockdown به صورت زیر می باشند:

- جلوگیری از تغییرات خطرناک در Kernel
- مکمل Secure Boot

نقاط ضعف Kernel Lockdown به صورت زیر می باشند:

- برخی ابزارهای توسعه و دیباگ را محدود می کند.

بررسی ها نشان می دهد که استفاده از مکانیزم های امنیتی سخت افزاری و مبتنی بر بوت در سال های اخیر رشد چشمگیری داشته است. یکی از مهم ترین دلایل این رشد، الزامات امنیتی ویندوز 11 است که وجود TPM 2.0 و پشتیبانی از Secure Boot را برای نصب سیستم عامل الزامی کرده است. امروزه تقریباً تمام رایانه های جدید مبتنی بر UEFI از Secure Boot پشتیبانی می کنند و اکثر سیستم های تولید شده در چند سال اخیر نیز به TPM 2.0 مجهز هستند. در عمل می توان گفت که TPM و Secure Boot از یک قابلیت اختیاری به یک استاندارد رایج در صنعت تبدیل شده اند. از دید امنیتی، ترکیب Secure Boot، TPM، LUKS، ASLR و NX می تواند بخش بزرگی از حملات رایج را خنثی یا بسیار دشوار کند. به همین دلیل رویکرد فعلی صنعت استفاده از دفاع چندلایه است، به گونه ای که اگر یک لایه امنیتی دور زده شود، لایه های دیگر همچنان از سیستم محافظت کنند.

سوال ۲

Secure Boot مکانیزمی است که هنگام راه اندازی سیستم، اطمینان حاصل می کند فقط نرم افزارهای راه انداز و اجزایی اجرا می شوند که توسط یک مرجع مورد اعتماد امضا شده باشند. در این فرایند، میان افزار دستگاه ابتدا امضای دیجیتال برنامه راه انداز را بررسی می کند. اگر امضا معتبر باشد، اجرای آن ادامه پیدا می کند. در غیر این صورت، از اجرای آن جلوگیری می شود. به این ترتیب، بدافزارها یا کدهای غیرمجاز نمی توانند در مراحل اولیه راه اندازی سیستم اجرا شوند.

Measured Boot علاوه بر بررسی اعتبار اجزا، اطلاعات مربوط به هر جزء بارگذاری شده را نیز ثبت می کند. در هر مرحله، یک مقدار هش از جزء موردنظر محاسبه شده و در یک سخت افزار امن مانند تراشه امنیتی ذخیره می شود. در پایان فرایند راه اندازی، مجموعه این اندازه گیری ها تصویری از وضعیت واقعی سیستم ارائه می دهد. بنابراین می توان بعدا بررسی کرد که دقیقا چه اجزایی اجرا شده اند و آیا تغییری در آن ها رخ داده است یا خیر.

در سیستم دارای Secure Boot بدون Measured Boot، فقط از اجرای اجزای فاقد امضای معتبر جلوگیری می شود، اما سابقه ای از آنچه واقعا اجرا شده نگهداری نمی شود. اگر مهاجم بتواند به یک نسخه قدیمی اما همچنان دارای امضای معتبر دسترسی پیدا کند، ممکن است آن نسخه را جایگزین نسخه جدید کند. از آنجا که امضا معتبر است، راه اندازی امن مانع اجرای آن نمی شود. در چنین شرایطی، سیستم ممکن است آسیب پذیر شود، در حالی که هیچ مکانیزمی برای ثبت و تشخیص این تغییر وجود ندارد.

همچنین در نبود Measured Boot، مدیر سیستم یا سامانه های نظارتی نمی توانند پس از روشن شدن دستگاه ثابت کنند که فرایند راه اندازی دقیقا مطابق انتظار انجام شده است. در نتیجه، توانایی کشف دستکاری های پیچیده یا بازگردانی به نسخه های قدیمی کاهش می یابد.

تفاوت اصلی این دو مکانیزم در هدف آن هاست. Secure Boot بر جلوگیری از اجرای کدهای غیرمجاز تمرکز دارد، در حالی که Measured Boot بر ثبت و گزارش وضعیت واقعی فرایند راه اندازی تمرکز می کند. به طور کلی، Secure Boot تضمین می کند که فقط اجزای مورد اعتماد اجرا شوند، اما نشان نمی دهد چه چیزی واقعا اجرا شده است. در مقابل، Measured Boot امکان بررسی و اثبات وضعیت واقعی سیستم را فراهم می کند. استفاده همزمان از هر دو مکانیزم امنیت بالاتری ایجاد می کند، زیرا هم از اجرای کدهای غیرمجاز جلوگیری می شود و هم هرگونه تغییر یا رفتار غیرمنتظره قابل شناسایی خواهد بود.

سوال ۳

systemd-cryptenroll و systemd-pcrlock دو ابزار از مجموعه systemd هستند که برای استفاده از TPM 2.0 و افزایش امنیت فرایند بوت طراحی شده‌اند و یکی از اهداف مهم آن‌ها، کاهش ریسک حملات Evil Maid است.

systemd-cryptenroll ابزاری است که کلید بازکردن پارتیشن رمزگذاری شده از دیسک را به FIDO2 Token، TPM یا Smart Card متصل می‌کند. در سناریوی TPM، کلید رمزگشایی دیسک داخل TPM محافظت می‌شود و تنها زمانی آزاد می‌گردد که مقادیر مشخصی از PCR با وضعیت مورد انتظار مطابقت داشته باشند. به بیان ساده، هنگام راه‌اندازی، TPM وضعیت اجزای بوت را اندازه‌گیری می‌کند کرده و نتایج را در PCR ها ذخیره می‌کند. اگر بوت‌لودر، کرنل یا Secure Boot تغییر کرده باشد، مقادیر PCR عوض می‌شوند. در این حالت TPM کلید را آزاد نمی‌کند. بنابراین مهاجم نمی‌تواند به‌سادگی یک بوت‌لودر دستکاری شده نصب کند و انتظار داشته باشد دیسک به‌صورت خودکار باز شود. این ویژگی باعث می‌شود تا بسیاری از حملات Evil Maid که بر تغییر زنجیره بوت متکی هستند، شکست بخورند.

نقاط قوت systemd-cryptenroll به صورت زیر می‌باشند:

- استفاده از آن نسبتاً ساده است.
- نیازی به نگهداری کلید رمزگشایی روی دیسک وجود ندارد.
- در صورت استفاده همراه با Secure Boot، زنجیره اعتماد نسبتاً محکمی ایجاد می‌کند.
- در برابر بسیاری از تغییرات غیرمجاز در فرایند بوت محافظت ایجاد می‌کند.
- برای استفاده روزمره بسیار راحت است زیرا کاربر معمولاً نیازی به وارد کردن رمز عبور ندارد.

نقاط ضعف systemd-cryptenroll به صورت زیر می‌باشند:

- وابستگی شدید به انتخاب PCR های مناسب دارد.
- اگر PCR های کمی انتخاب شوند، ممکن است برخی تغییرات مخرب شناسایی نشوند.
- اگر PCR های زیادی انتخاب شوند، هر به‌روزرسانی کرنل یا بوت‌لودر می‌تواند باعث از کار افتادن بازشدن خودکار دیسک شود.
- این ابزار صرفاً بررسی می‌کند که وضعیت فعلی با وضعیت ثبت‌شده یکسان باشد و خودش مکانیزمی برای تعریف سیاست‌های پیچیده یا نسخه‌های مجاز متعدد ندارد.
- در صورت سرقت فیزیکی کامل دستگاه، اگر مهاجم بتواند همان وضعیت بوت مورد انتظار را بازسازی کند، TPM همچنان کلید را آزاد خواهد کرد.

systemd-pcrlock ابزار جدیدتری است که محدودیت‌های روش سنتی PCR Binding را برطرف می‌کند. این ابزار به جای قفل کردن کلید روی یک مجموعه مقدار ثابت PCR، امکان تعریف سیاست‌های مجاز را فراهم می‌کند و در عمل می‌توان مشخص کرد که چندین وضعیت مختلف بوت معتبر هستند. برای مثال می‌توان موارد زیر را تعیین نمود:

- کرنل نسخه 6.10 مجاز است.
- کرنل نسخه 6.11 مجاز است.
- نسخه جدید Secure Boot مجاز است.

Evil Maid معمولاً مهاجم سعی می‌کند مؤلفه‌های بوت را تغییر دهد. systemd-pcrlock مجموعه‌ای از وضعیت‌های معتبر را تعریف می‌کند و هر وضعیتی خارج از این مجموعه رد می‌شود. در نتیجه مواردی مانند بوت‌لودر دستکاری شده، کرنل ناشناس، initramfs مخرب، تغییرات غیرمجاز Secure Boot و ... رد می‌شوند. در عین حال، به‌روزرسانی‌های قانونی سیستم همچنان می‌توانند در فهرست وضعیت‌های مجاز قرار گیرند.

نقاط قوت systemd-pcrlock به صورت زیر می‌باشند:

- انعطاف‌پذیری بسیار بیشتری نسبت به PCR Binding سنتی دارد.
- به‌روزرسانی کرنل و سیستم را آسان‌تر می‌کند.
- می‌تواند چندین وضعیت بوت معتبر را همزمان پشتیبانی کند.
- احتمال قفل شدن ناخواسته سیستم پس از آپدیت را کاهش می‌دهد.
- برای سیستم‌هایی که مرتب به‌روزرسانی می‌شوند، مناسب‌تر است.

نقاط ضعف systemd-pcrlock به صورت زیر می‌باشند:

- تنظیم آن پیچیده‌تر از systemd-cryptenroll است.
- درک و مدیریت سیاست‌های PCR نیازمند آشنایی بیشتری با TPM و Secure Boot است.
- در صورت تعریف اشتباه سیاست‌ها، ممکن است وضعیت‌های نا امن نیز به‌عنوان مجاز پذیرفته شوند.
- ابزار نسبتاً جدیدی است و تجربه عملیاتی آن به اندازه روش‌های سنتی گسترده نیست.

در برابر مهاجم Evil Maid، وضعیت امنیتی سیستم‌های امروزی را باید ترکیبی از پیشرفت قابل توجه و باقی ماندن ضعف‌های بنیادی دانست. در حالت واقع‌بینانه، اگر مهاجم دسترسی فیزیکی کوتاه‌مدت و بدون نظارت داشته باشد، هنوز هم می‌تواند در بسیاری از سیستم‌های رایج به سطحی از نفوذ برسد که عملاً امنیت نرم‌افزارهای بالادستی را بی‌اثر کند. دلیل اصلی این است که زنجیره اعتماد از لحظه روشن شدن دستگاه شروع می‌شود، و اگر همان ابتدا در firmware یا boot chain دستکاری انجام شود، سیستم‌عامل و ابزارهای امنیتی بعدی دیگر نمی‌توانند به‌طور کامل به محیط اعتماد کنند. حتی در حضور رمزگذاری کامل دیسک، اگر کلیدها در مرحله بوت آزاد شوند یا شرایط اعتماد به‌درستی سنجیده نشود، مهاجم می‌تواند به داده‌ها یا حداقل به مسیر اجرای سیستم دسترسی پیدا کند.

با این حال، وضعیت نسبت به گذشته بسیار بهتر شده است. ترکیب TPM، Secure Boot، و measured boot یک مدل دفاعی جدی ایجاد کرده که در صورت درست پیاده‌سازی، حمله را از حالت بی‌هزینه و سریع به یک حمله پیچیده، زمان‌بر و پر ریسک تبدیل می‌کند. وقتی کلیدهای رمزگذاری به وضعیت سخت‌افزار و اندازه‌گیری بوت گره می‌خورند، هر تغییر کوچک در boot chain می‌تواند باعث قفل شدن دسترسی به داده‌ها شود. این یعنی مهاجم دیگر نمی‌تواند بدون ردپا یا بدون از دست دادن دسترسی، سیستم را تغییر دهد. علاوه بر آن، attestation در سیستم‌های سازمانی امکان تشخیص تغییرات را فراهم می‌کند، و همین موضوع ارزش حمله را کاهش می‌دهد. قفل کردن UEFI، غیرفعال کردن بوت خارجی، کنترل فیزیکی پورت‌ها، و استفاده از دیسک‌های رمزگذاری شده با تنظیمات مقاوم در برابر دستکاری نیز این لایه دفاعی را تقویت می‌کنند.

در سیستم‌های پیشرفته‌تر، استفاده از ماژول‌های سخت‌افزاری امن و معماری‌هایی که حداقل اعتماد را به firmware می‌دهند، می‌تواند سطح امنیت را بالاتر ببرد. حتی پروژه‌هایی مثل coreboot یا سیستم‌های دارای طراحی بازتر در سطح firmware، امکان بررسی و کاهش سطح پنهان‌کاری را فراهم می‌کنند. اما این راهکارها هنوز عمومی نشده‌اند و در بسیاری از لپ‌تاپ‌ها و مادربردهای رایج، کاربر عملاً به firmware بسته وابسته است.

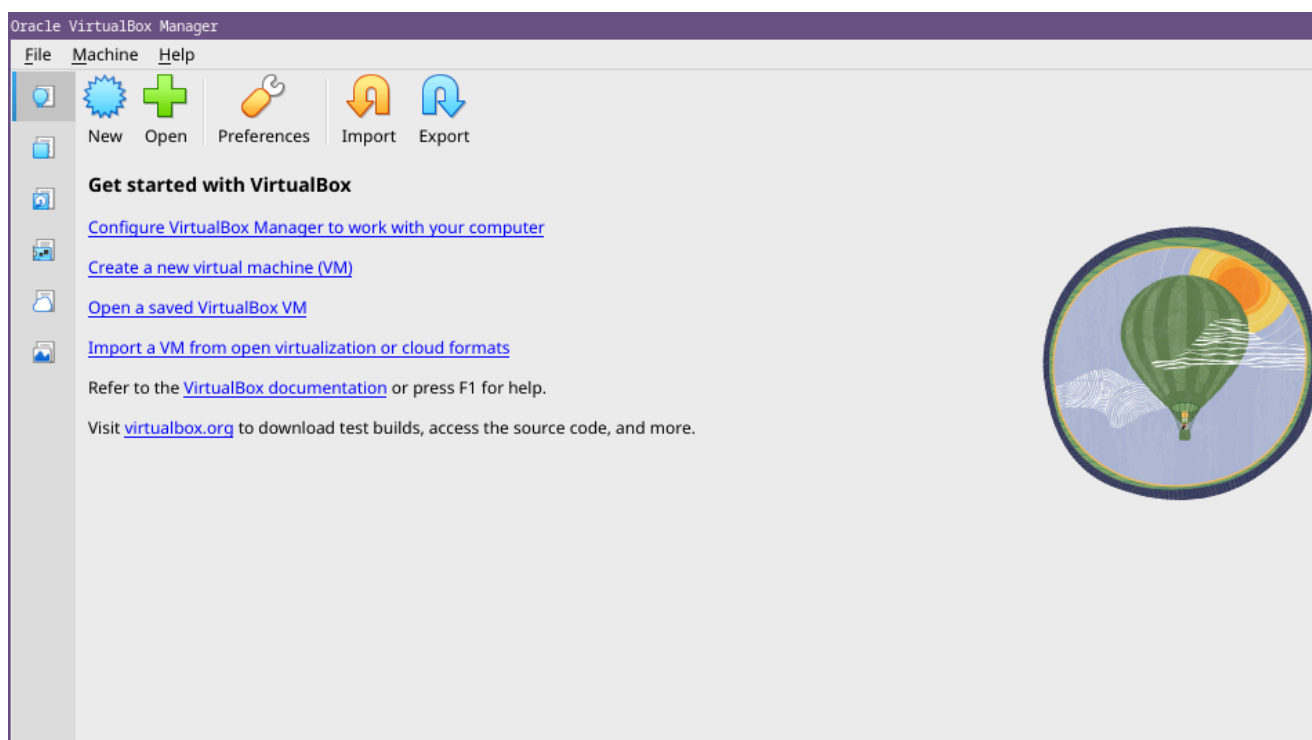
با وجود این پیشرفت‌ها، شکاف‌های مهم همچنان باقی مانده‌اند. اول، پیچیدگی زنجیره اعتماد است. هرچه لایه‌های بیشتری اضافه می‌شود، احتمال خطای پیکربندی یا ضعف پیاده‌سازی هم بالا می‌رود. در عمل، بسیاری از سیستم‌ها Secure Boot یا TPM را دارند، اما به‌درستی یا با تنظیمات سخت‌گیرانه استفاده نمی‌کنند، که سطح امنیت واقعی را کاهش می‌دهد. دوم، حملات سطح پایین‌تر از boot chain همچنان فعال هستند، مانند حملات DMA از طریق Thunderbolt یا PCIe، حملات cold boot برای استخراج کلید از RAM، و سوءاستفاده از SMM که خارج از کنترل سیستم‌عامل اجرا می‌شود. سوم، firmware و microcode پردازنده‌ها معمولاً بسته و غیرقابل ممیزی هستند، و این یعنی حتی متخصصان هم دید کامل به رفتار واقعی سیستم ندارند.

نکته مهم دیگر، مسئله زنجیره تأمین است. حتی اگر کاربر همه تنظیمات را درست انجام دهد، اعتماد به سخت افزار تولیدشده در کارخانه همچنان یک نقطه مرکزی شکست است. اگر در سطح تولید یا توزیع تغییراتی ایجاد شده باشد، تشخیص آن برای کاربر نهایی بسیار دشوار یا غیرممکن است. در نهایت، مسئله usability و واقعیت عملی نیز تعیین کننده است: بسیاری از این دفاع ها یا نیاز به دانش تخصصی دارند یا باعث پیچیدگی در استفاده روزمره می شوند، و همین باعث می شود در دنیای واقعی، سطح امنیت عملی پایین تر از سطح نظری باقی بماند.

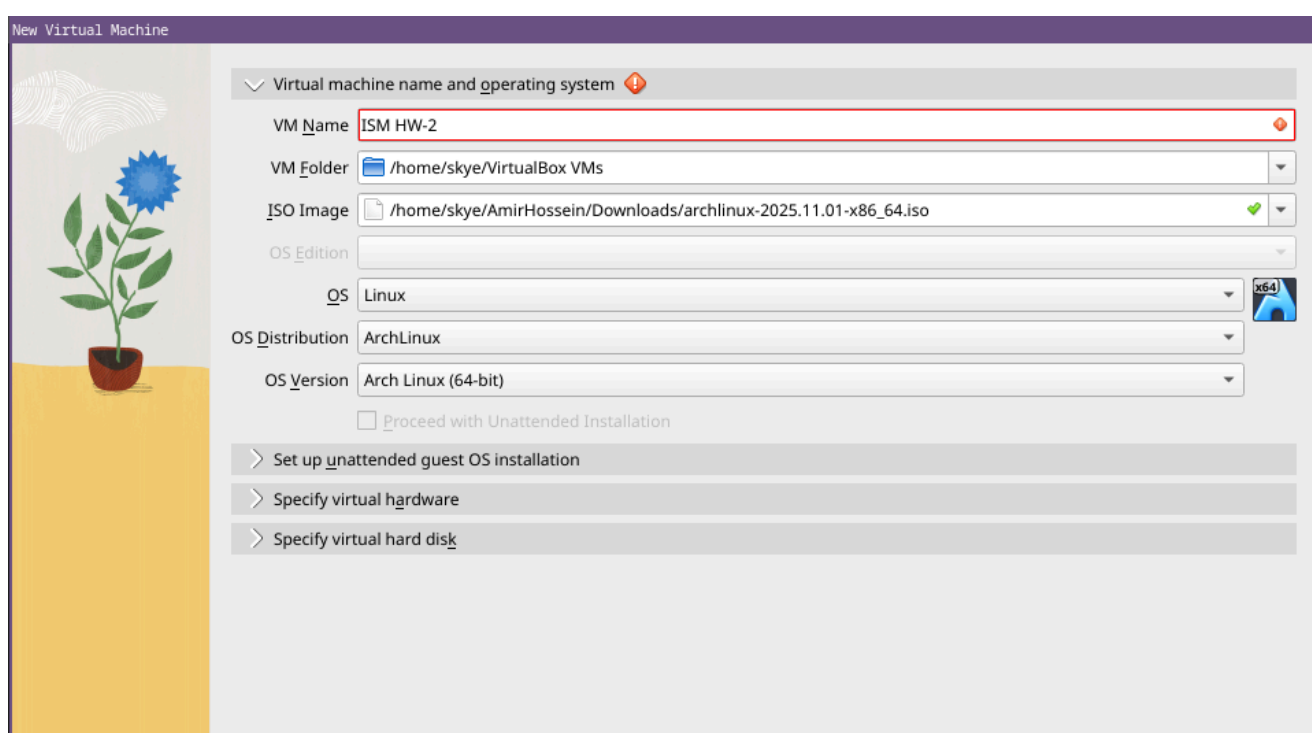
در نتیجه در برابر مهاجم مصمم، امنیت غیرقابل نفوذ نداریم، بلکه فقط هزینه و پیچیدگی حمله را بالا برده ایم. تفاوت اصلی این است که مهاجم دیگر به یک دسترسی کوتاه و ساده برای نفوذ پایدار قانع نمی شود، بلکه باید وارد حوزه firmware، سخت افزار و زنجیره اعتماد شود، جایی که هم سخت تر است و هم ردگیری آن دشوارتر، اما همچنان کاملاً ممکن است.

بخش ۳.۱

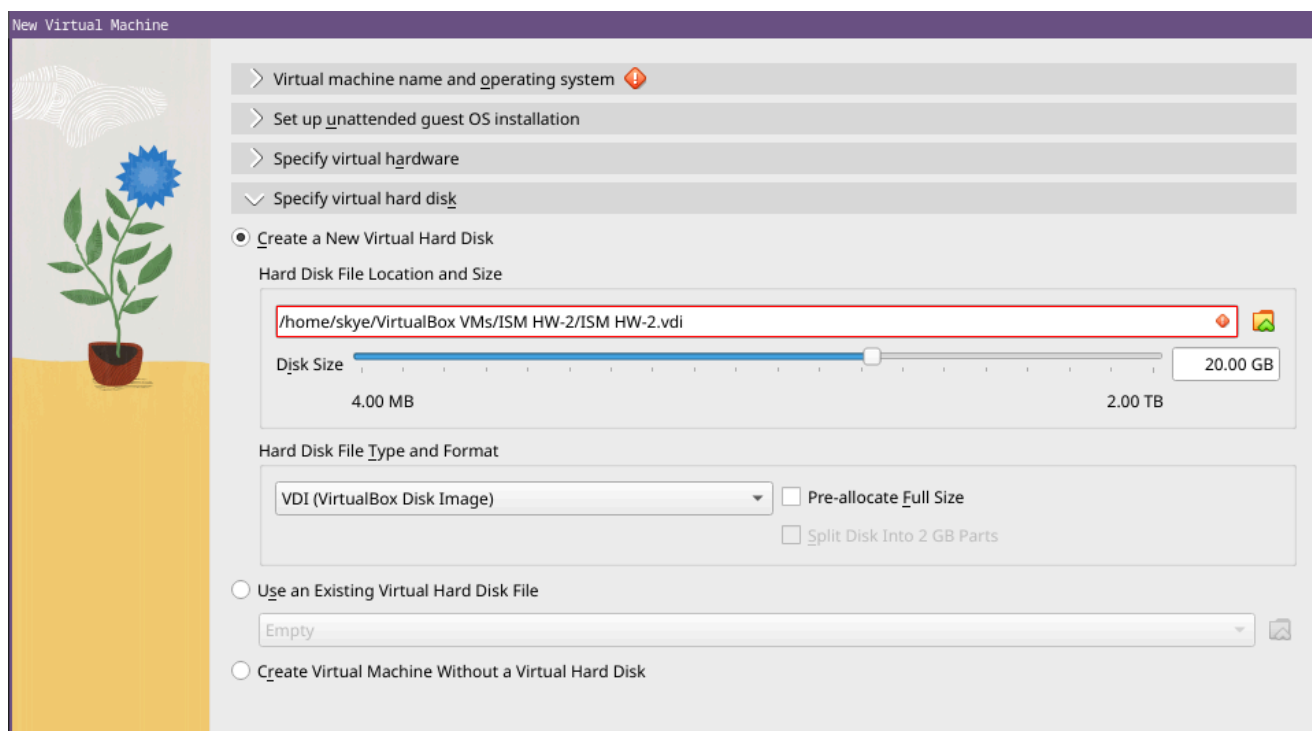
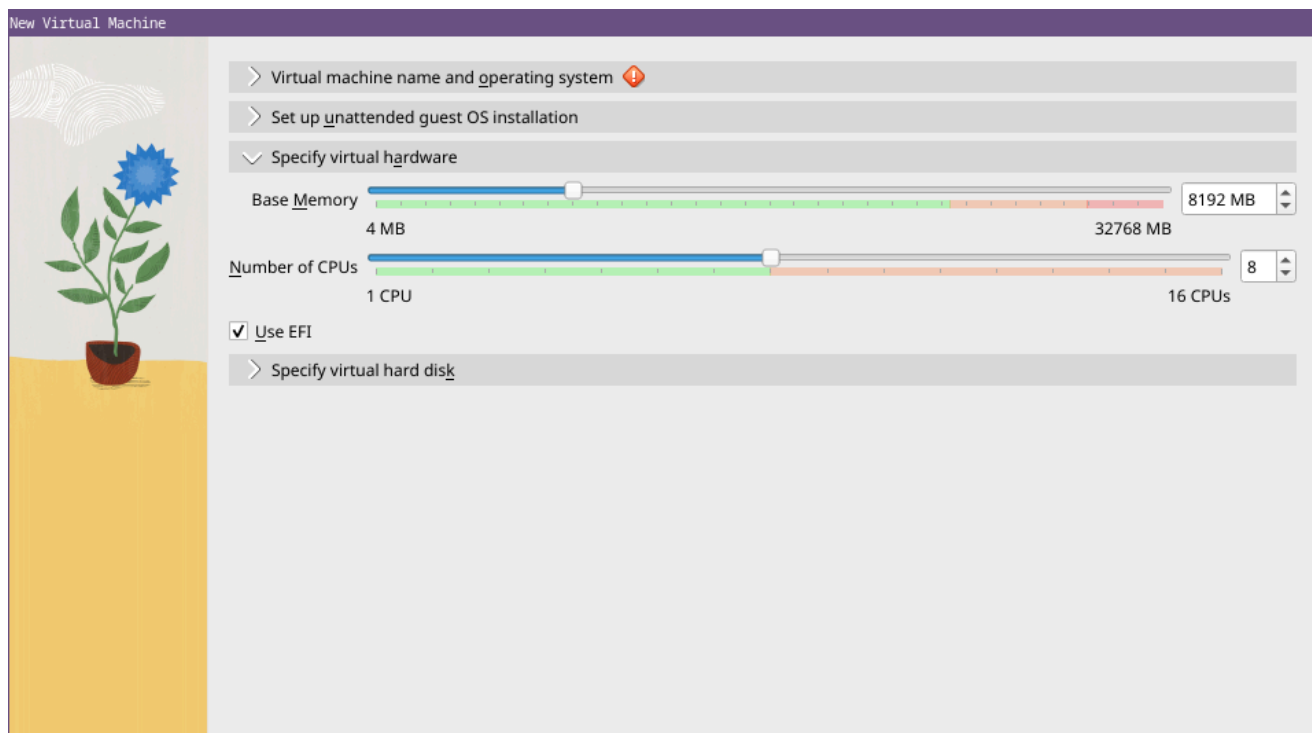
ابتدا نیاز است تا یک ماشین مجازی دارای سیستم عامل arch ایجاد شده و راه اندازی شود.
برای این منظور، از Oracle VirtualBox استفاده شده است. محیط اولیه این ابزار به صورت زیر می باشد.



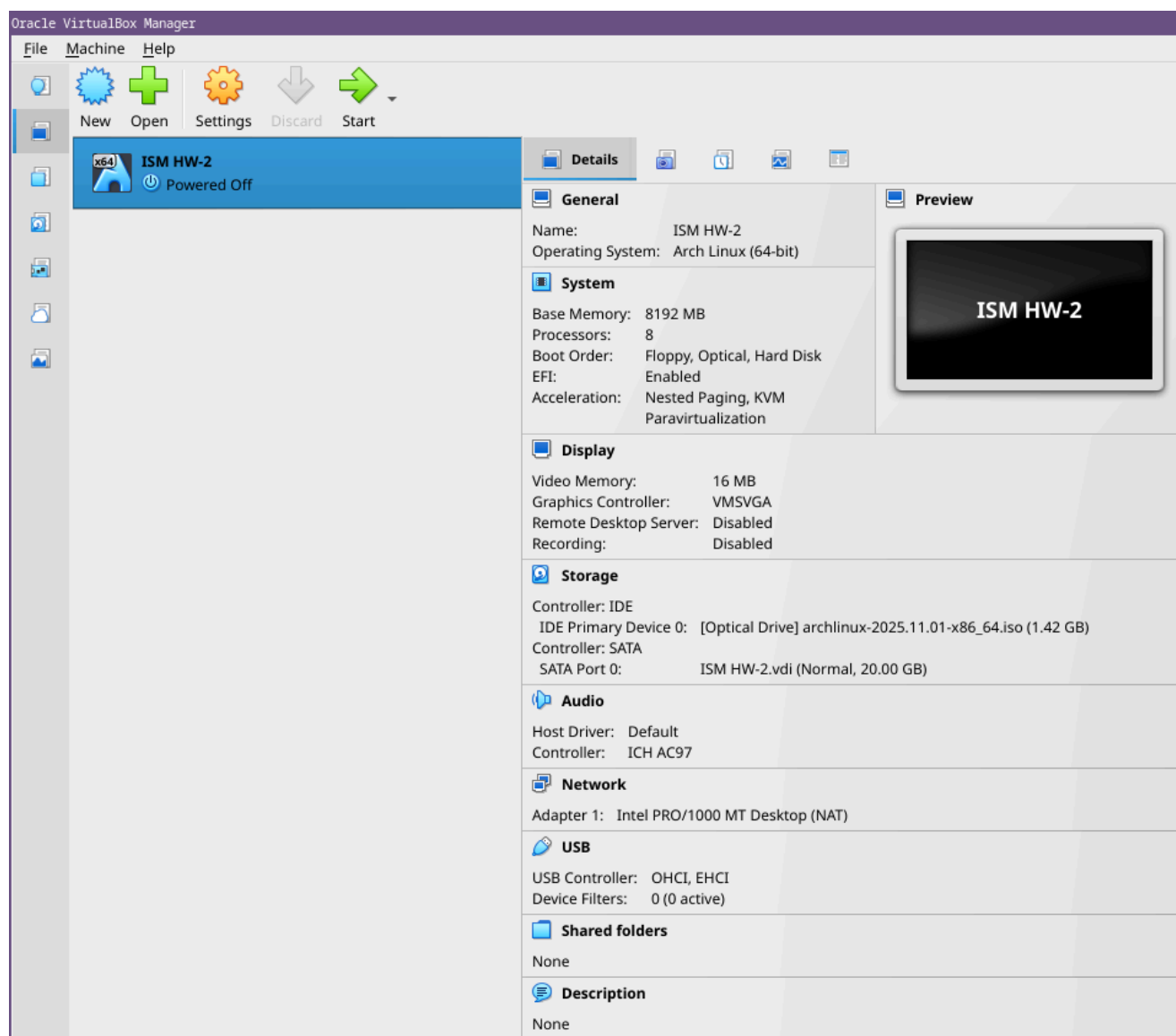
ابتدا نیاز است تا نام ماشین مجازی و iso مورد استفاده برای راه اندازی آن مشخص شوند.



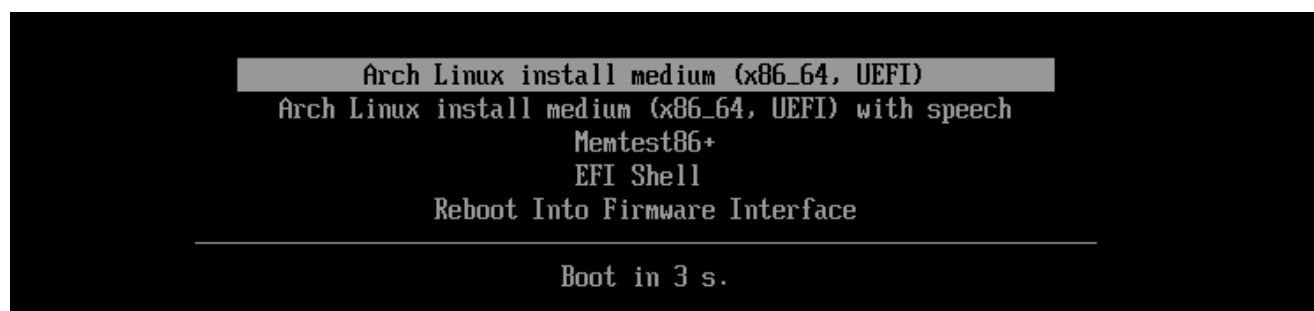
در ادامه منابع ماشین مجازی تعیین می‌شوند. برای این ماشین مجازی، مقدار 8GB رم، 8 هسته CPU و 20GB حافظه اختصاص داده شده است. همچنین تیک مربوط به گزینه، EFI نیز زده شده است تا سیستم عامل در حالت UEFI راه اندازی شود.



اطلاعات ماشین مجازی ایجاد شده در تصویر زیر قابل مشاهده می باشد.



با راه اندازی ماشین مجازی ایجاد شده، صفحه مربوط به بوت شدن iso سیستم عامل arch نمایش داده خواهد شد. با انتخاب اولین گزینه، live image مربوطه load شده و یک ترمینال نمایش داده می شود.



ترمینال نمایش داده به صورت زیر می باشد. در ادامه باید تنظیمات مربوط به سیستم عامل arch به منظور استفاده از آن صورت بگیرد.

```
Arch Linux 6.17.6-arch1-1 (tty1)

Try contacting this VM's SSH server via 'ssh vsock%1' from host.

archiso login: root (automatic login)

To install Arch Linux follow the installation guide:
https://wiki.archlinux.org/title/Installation_guide

For Wi-Fi, authenticate to the wireless network using the iwctl utility.
For mobile broadband (WWAN) modems, connect with the mmcli utility.
Ethernet, WLAN and WWAN interfaces using DHCP should work automatically.

After connecting to the internet, the installation guide can be accessed
via the convenience script Installation_guide.

2
root@archiso ~ #
root@archiso ~ #
```

ابتدا بررسی شده است که سیستم عامل در حالت UEFI راه اندازی شده باشد. این عمل با استفاده از نمایش محتویات فایل `/sys/firmware/efi/fw_platform_size` صورت گرفته است. مقدار نمایش داده شده نشان می دهد که سیستم عامل در حالت UEFI 64-bit راه اندازی شده است.

```
root@archiso ~ #
root@archiso ~ # cat /sys/firmware/efi/fw_platform_size
64
root@archiso ~ #
```

در ادامه تنظیمات مربوط به تاریخ، ساعت و timezone سیستم عامل انجام شده است. با توجه به تصویر زیر، ابتدا ساعت بر روی UTC +0000 تنظیم شده بوده و بعد از تعیین timezone، این مقدار به UTC +0330 تغییر پیدا کرده است.

```
root@archiso ~ #
root@archiso ~ # timedatectl
          Local time: Fri 2026-05-29 17:35:35 UTC
          Universal time: Fri 2026-05-29 17:35:35 UTC
              RTC time: Fri 2026-05-29 17:35:35
              Time zone: UTC (UTC, +0000)
System clock synchronized: yes
          NTP service: active
          RTC in local TZ: no
root@archiso ~ # timedatectl set-timezone Asia/Tehran
root@archiso ~ # timedatectl
          Local time: Fri 2026-05-29 21:05:39 +0330
          Universal time: Fri 2026-05-29 17:35:39 UTC
              RTC time: Fri 2026-05-29 17:35:40
              Time zone: Asia/Tehran (+0330, +0330)
System clock synchronized: yes
          NTP service: active
          RTC in local TZ: no
root@archiso ~ # _
```

برای راه اندازی سیستم عامل نیاز است تا فضای ذخیره سازی پارتیشن بندی شده و فرمت شود. دیسک های در اختیار ماشین مجازی، به صورت زیر می باشند. همانطور مشاهده می شود، دیسکی که دارای حجم 20GB هست، همان virtual disk است که در ابتدا به ماشین مجازی تخصیص داده شد.

```
root@archiso ~ #
root@archiso ~ # fdisk -l
Disk /dev/sda: 20 GiB, 21474836480 bytes, 41943040 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes

Disk /dev/loop0: 961.48 MiB, 1008189440 bytes, 1969120 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
root@archiso ~ # _
```

برای پارتیشن بندی این دیسک، از ابزار fdisk استفاده شده است. برای این ماشین مجازی دو پارتیشن برای boot و root در نظر گرفته شده است تا پیچیدگی تا جای ممکن کاهش پیدا کند.

```
root@archiso ~ #
root@archiso ~ # fdisk /dev/sda

Welcome to fdisk (util-linux 2.41.2).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Device does not contain a recognized partition table.
Created a new DOS (MBR) disklabel with disk identifier 0x71d71e1a.

Command (m for help):
```

ابتدا با ورودی دادن g، یک disk label جدید GPT ایجاد شده است. اینکار برای استفاده از EFI System Partition ضروری می باشد.

```
Command (m for help): g
Created a new GPT disklabel (GUID: C5A983BC-EFFD-4ADA-A7C2-B8D4879EADD).

Command (m for help): _
```

در ادامه با ورودی دادن n، یک پارتیشن جدید با حجم 512MB ایجاد شده و نوع آن با استفاده از ورودی دادن t، روی EFI System تنظیم شده است. این پارتیشن مربوط به boot می باشد.

```
Command (m for help): n
Partition number (1-128, default 1):
First sector (2048-41943006, default 2048):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-41943006, default 41940991): +512M

Created a new partition 1 of type 'Linux filesystem' and of size 512 MiB.

Command (m for help): t
Selected partition 1
Partition type or alias (type L to list all): 1
Changed type of partition 'Linux filesystem' to 'EFI System'.

Command (m for help):
```

دوباره با ورودی دادن n، یک پارتیشن جدید با تمام حجم باقی مانده از دیسک ایجاد شده است. این پارتیشن مربوط به root می باشد.

```
Command (m for help): n
Partition number (2-128, default 2):
First sector (1050624-41943006, default 1050624):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (1050624-41943006, default 41940991):

Created a new partition 2 of type 'Linux filesystem' and of size 19.5 GiB.
Command (m for help): _
```

در نهایت نیز با ورودی دادن w، تنظیمات صورت گرفته اعمال شده و پارتیشن بندی دیسک تکمیل می گردد.

```
Command (m for help): w
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.
```

در ادامه boot و root به ترتیب با استفاده از fat32 و ext4 فرمت شده اند.

```
root@archiso ~ #
root@archiso ~ # mkfs.fat -F32 /dev/sda1
mkfs.fat 4.2 (2021-01-31)
root@archiso ~ # mkfs.ext4 /dev/sda2
mke2fs 1.47.3 (8-Jul-2025)
Creating filesystem with 5111296 4k blocks and 1277952 inodes
Filesystem UUID: 0c50fabd-0685-4388-bb64-fb6054f4dfef
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632, 2654208,
    4096000

Allocating group tables: done
Writing inode tables: done
Creating journal (32768 blocks): done
Writing superblocks and filesystem accounting information: done

root@archiso ~ #
```

حالا می توان پارتیشن مربوط به root را در مسیر /mnt و پارتیشن مربوط به boot را در مسیر /mnt/boot به ترتیب mount نمود.

```
root@archiso ~ #
root@archiso ~ # mount /dev/sda2 /mnt
root@archiso ~ # mount --mkdir /dev/sda1 /mnt/boot
root@archiso ~ #
```

در ادامه نیاز است تا یک سری پکیج‌های اولیه و ضروری برای سیستم‌عامل جدید نصب شوند.

```
root@archiso ~ #
root@archiso ~ # pacstrap -K /mnt base linux linux-firmware
==> Creating install root at /mnt
gpg: /mnt/etc/pacman.d/gnupg/trustdb.gpg: trustdb created
gpg: no ultimately trusted keys found
gpg: starting migration from earlier GnuPG versions
gpg: porting secret keys from '/mnt/etc/pacman.d/gnupg/secring.gpg' to gpg-agent
gpg: migration succeeded
==> Generating pacman master key. This may take some time.
gpg: Generating pacman keyring master key...
gpg: directory '/mnt/etc/pacman.d/gnupg/openpgp-revocs.d' created
gpg: revocation certificate stored as '/mnt/etc/pacman.d/gnupg/openpgp-revocs.d/8699C6A6A0D1F2CFE8B341C4420CF451162E861'
gpg: Done
==> Updating trust database...
gpg: marginals needed: 3 completes needed: 1 trust model: pgp
gpg: depth: 0 valid: 1 signed: 0 trust: 0-, 0q, 0n, 0m, 0f, 1u
==> Installing packages to /mnt
:: Synchronizing package databases...
core                               126.1 KiB   65.6 KiB/s 00:02 [#####]
extra                             2.8 MiB    895 KiB/s 00:06 [#####]----
```

با استفاده از دستور زیر یک فایل fstab برای بوت شدن سیستم‌عامل ایجاد شده است.

```
root@archiso ~ #
root@archiso ~ # genfstab -U /mnt >> /mnt/etc/fstab
root@archiso ~ #
```

حالا می‌توان با استفاده از دستور arch-chroot وارد محیط سیستم‌عامل جدید شده و با آن تعامل داشت.

```
root@archiso ~ #
root@archiso ~ # arch-chroot /mnt
[root@archiso /]# ls
bin boot dev etc home lib lib64 lost+found mnt opt proc root run sbin srv sys tmp usr var
[root@archiso /]# _
```

در ادامه تنظیمات مربوط به timezone، تاریخ، ساعت، و نام host انجام شده است تا سیستم‌عامل آماده استفاده شود. بدین صورت تنظیمات اولیه مربوط به نصب سیستم‌عامل arch انجام شده و می‌توان وارد بخش بعدی به منظور تولید UKI شد.

```
[root@archiso /]#
[root@archiso /]# ln -sf /usr/share/zoneinfo/Asia/Tehran /etc/localtime
[root@archiso /]# hwclock --systohc
[root@archiso /]# _
```

```
[root@archiso /]#
[root@archiso /]# echo "en_US.UTF-8 UTF-8" > /etc/locale.gen
[root@archiso /]# locale-gen
Generating locales...
en_US.UTF-8... done
Generation complete.
[root@archiso /]# echo "LANG=en_US.UTF-8" > /etc/locale.conf
[root@archiso /]# _
```

```
[root@archiso /]#
[root@archiso /]# echo "skye" > /etc/hostname
[root@archiso /]#
```


برای وصل شدن به اینترنت در آینده، ایجاد یک UKI و بوت کردن آن، ابتدا نیاز است تا پکیج‌های زیر نصب شوند.

```
[root@archiso /]#
[root@archiso /]# pacman -Sy networkmanager efibootmgr mkinitcpio base-devel ukify systemd sbctl
:: Synchronizing package databases...
core
extra
```

در ادامه نیاز است تا initramfs بار دیگر برای اعمال تغییرات صورت گرفته، ساخته شود. این کار با استفاده از دستور mkinitcpio صورت گرفته است.

```
[root@archiso /]#
[root@archiso /]# mkinitcpio -P
==> Building image from preset: /etc/mkinitcpio.d/linux.preset: 'default'
==> Using default configuration file: '/etc/mkinitcpio.conf'
-> -k /boot/vmlinuz-linux -g /boot/initramfs-linux.img
==> Starting build: '7.0.10-arch1-1'
-> Running build hook: [base]
-> Running build hook: [systemd]
-> Running build hook: [autodetect]
-> Running build hook: [microcode]
-> Running build hook: [modconf]
-> Running build hook: [kms]
-> Running build hook: [keyboard]
-> Running build hook: [sd-vconsole]
==> WARNING: sd-vconsole: "/etc/vconsole.conf" not found, will use default values
-> Running build hook: [block]
-> Running build hook: [filesystems]
-> Running build hook: [fsck]
==> Generating module dependencies
==> Creating zstd-compressed initcpio image: '/boot/initramfs-linux.img'
-> Early uncompressed CPIO image generation successful
==> Initcpio image generation successful
==> Running post hooks
-> Running post hook: [sbctl]
Secureboot key directory doesn't exist, not signing!
==> Post processing done
[root@archiso /]# _
```

با استفاده از دستور زیر نیز، systemd bootloader بر روی EFI System Partition نصب شده است تا ماشین مجازی بتواند با استفاده از UEFI بوت شود.

```
[root@archiso /]#
[root@archiso /]# bootctl install
Running in a chroot, enabling --graceful.
Created directory "/boot/EFI".
Created directory "/boot/EFI/systemd".
Created directory "/boot/EFI/BOOT".
Created directory "/boot/loader".
Created directory "/boot/loader/keys".
Created directory "/boot/loader/entries".
Created directory "/boot/EFI".
Created directory "/boot/EFI/Linux".
Copied "/usr/lib/systemd/boot/efi/systemd-bootx64.efi" to "/boot/EFI/systemd/systemd-bootx64.efi".
Copied "/usr/lib/systemd/boot/efi/systemd-bootx64.efi" to "/boot/EFI/BOOT/BOOTX64.EFI".
```

برای ساخت UKI از ابزار ukify استفاده شده است که فایل arch.efi را در مسیر /boot/EFI/Linux/arch.efi ایجاد می‌کند.

```
[root@archiso /]#
[root@archiso /]# ukify build \
> --linux /boot/vmlinuz-linux \
> --initrd /boot/initramfs-linux.img \
> --initrd /boot/intel-ucode.img \
> --cmdline "root=UUID=$(blkid -s UUID -o value /dev/sda2) rw" \
> --output /boot/EFI/Linux/arch.efi
Kernel version not specified, starting autodetection ■ .
Found uname version: 7.0.10-arch1-1
Wrote unsigned /boot/EFI/Linux/arch.efi
[root@archiso /]# _
```

در ادامه با استفاده از ابزار efibootmgr می‌توان فایل arch.efi ساخته شده را به عنوان یکی از اولویت‌های بوت در زمان راه‌اندازی قرار داد.

```
[root@archiso /]#
[root@archiso /]# efibootmgr -c -d /dev/sda -p 1 -L "Arch UKI" -l "\EFI\Linux\arch.efi"
BootCurrent: 0001
Timeout: 0 seconds
BootOrder: 0003,0000,0001,0002
Boot0000* UiaApp FvUol(7cb8bdc9-f8eb-4f34-aaea-3ee4af6516a1)/FvFile(462caa21-7614-4503-836e-8ab6f4662331)
Boot0001* UEFI VBOX CD-ROM VBO-01f003f6 PciRoot(0x0)/Pci(0x1,0x1)/Ata(0,0,0){auto_created_boot_option}
Boot0002* UEFI VBOX HARDDISK VBF5825ef9-9b05fa18 PciRoot(0x0)/Pci(0xd,0x0)/Sata(0,65535,0){auto_created_boot_opti
Boot0003* Arch UKI HD(1,GPT,3a3c4c4b-89cc-4a52-8a3d-474dcef969be,0x800,0x100000)\EFI\Linux\arch.efi
[root@archiso /]# _
```

حالا می‌توان پس از تعیین رمز عبور، از محیط سیستم‌عامل که با استفاده از arch-chroot به آن وارد شده بودیم، خارج شده و سیستم‌عامل را دوباره راه‌اندازی نمود.

```
[root@archiso /]#
[root@archiso /]# exit
exit
arch-chroot /mnt 17.17s user 13.96s system 2% cpu 21:35.37 total
root@archiso ~ # umount -R /mnt
root@archiso ~ # reboot_
```

با راه‌اندازی مجدد سیستم‌عامل، می‌توان با استفاده از رمز عبور تعیین شده برای root، وارد محیط آن شد.

```
Arch Linux 7.0.10-arch1-1 (tty1)
skye login: root
Password:
[root@skye ~]#
```

همانطور که مشاهده می‌شود، محتویات فایل /proc/cmdline دارای خروجی مشخص شده در زمان ساخت UKI با استفاده از ابزار ukify بوده و efibootmgr نیز نشان می‌دهد که سیستم‌عامل از روی اولویت 0003 که همان UKI ایجاد شده در مسیر /boot/EFI/Linux/arch.efi بوت شده است.

```
[root@skye ~]#  
[root@skye ~]# cat /proc/cmdline  
root=UUID=0c50fabd-0685-4388-bb64-fb6054f4dfef rw  
[root@skye ~]# efibootmgr  
BootCurrent: 0003  
Timeout: 0 seconds  
BootOrder: 0003,0000,0001,0002  
Boot0000* UiaApp FvVol(7cb8bdc9-f8eb-4f34-aaea-3ee4af6516a1)/FvFile(462caa21-7614-4503-836e-8ab6f4662331)  
Boot0001* UEFI VBOX CD-ROM VBOX-01f003f6 PciRoot(0x0)/Pci(0x1,0x1)/Ata(0,0,0){auto_created_boot_option}  
Boot0002* UEFI VBOX HARDISK VBF5825ef9-9b05fa18 PciRoot(0x0)/Pci(0xd,0x0)/Sata(0,65535,0){auto_created_boot_opti  
Boot0003* Arch UKI HD(1,GPT,3a3c4c4b-89cc-4a52-8a3d-474dcef969be,0x800,0x100000)/\EFI\Linux\arch.efi  
[root@skye ~]# _
```

بخش ۳.۳

برای بیلد کردن و استفاده از ابزار ukibak ابتدا نیاز است تا کامپایلر زبان cargo، rust، و دیگر اجزای مورد نیاز این زبان نصب شوند. این عمل با استفاده از اسکریپتی که در مسیر زیر قرار دارد انجام شده است.

```
root@skye ~#  
root@skye ~# curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh  
info: downloading installer  
_
```

همانطور که مشاهده می‌شود، کامپایلر راسط و اجزای مورد نیاز آن با موفقیت نصب شده‌اند.

```
You can uninstall at any time with rustup self uninstall and  
these changes will be reverted.  
  
Current installation options:  
  
  default host triple: x86_64-unknown-linux-gnu  
    default toolchain: stable (default)  
        profile: default  
modify PATH variable: yes  
  
1) Proceed with standard installation (default - just press enter)  
2) Customize installation  
3) Cancel installation  
>  
  
info: profile set to default  
info: default host triple is x86_64-unknown-linux-gnu  
info: syncing channel updates for stable-x86_64-unknown-linux-gnu  
info: latest update on 2026-05-28 for version 1.96.0 (ac68faa20 2026-05-25)  
info: downloading 6 components  
   cargo installed                10.64 MiB  
  clippy installed                4.58 MiB  
 rust-docs installed             21.29 MiB  
 rust-std installed              28.36 MiB  
  rustc installed                76.47 MiB  
  rustfmt installed               2.06 MiB  
info: default toolchain set to stable-x86_64-unknown-linux-gnu  
  
  stable-x86_64-unknown-linux-gnu installed - rustc 1.96.0 (ac68faa20 2026-05-25)  
  
Rust is installed now. Great!
```

در ادامه نیاز است تا از مخزن ابزار ukibak در گیت‌هاب، کلون گرفته شود.

```
root@skye ~#  
root@skye ~# git clone https://github.com/FlashSystems/ukibak.git  
Cloning into 'ukibak' ...  
remote: Enumerating objects: 371, done.  
remote: Counting objects: 100% (35/35), done.  
remote: Compressing objects: 100% (14/14), done.  
remote: Total 371 (delta 24), reused 25 (delta 21), pack-reused 336 (from 1)  
Receiving objects: 100% (371/371), 666.18 KiB | 420.00 KiB/s, done.  
Resolving deltas: 100% (238/238), done.  
root@skye ~# cd ukibak/  
root@skye ~/ukibak (master)# _
```

برای بیلد کردن این ابزار، می‌توان از دستور `cargo build --release` استفاده نمود.

```
root@skye ~/ukibak (master)#
root@skye ~/ukibak (master)# cargo build --release
    Updating crates.io index
  Downloaded clap_derive v4.6.1
  Downloaded thiserror v2.0.18
  Downloaded simple_logger v5.2.0
  Downloaded const-fnv1a-hash v1.1.0
  Downloaded clap_builder v4.6.0
  Downloaded byteorder v1.5.0
  Downloaded clap_lex v1.1.0
  Downloaded quote v1.0.45
  Downloaded heck v0.5.0
  Downloaded thiserror-impl v2.0.18
  Downloaded clap v4.6.1
  Downloaded anstyle v1.0.14
  Downloaded log v0.4.29
  Downloaded libc v0.2.186
  Downloaded syn v2.0.117
  Downloaded unicode-ident v1.0.24
  Downloaded proc-macro2 v1.0.106
  Downloaded 17 crates (1.6MiB) in 6.04s
   Compiling proc-macro2 v1.0.106
   Compiling quote v1.0.45
   Compiling unicode-ident v1.0.24
   Compiling thiserror v2.0.18
   Compiling libc v0.2.186
   Compiling clap_lex v1.1.0
   Compiling heck v0.5.0
   Compiling anstyle v1.0.14
   Compiling log v0.4.29
   Compiling byteorder v1.5.0
   Compiling const-fnv1a-hash v1.1.0
   Compiling clap_builder v4.6.0
   Compiling simple_logger v5.2.0
   Compiling syn v2.0.117
   Compiling clap_derive v4.6.1
   Compiling thiserror-impl v2.0.18
   Compiling clap v4.6.1
   Compiling ukibak v1.1.5 (/root/ukibak)
   Finished `release` profile [optimized] target(s) in 22.82s
root@skye ~/ukibak (master)#
```

برای نصب ukibak نیز بر اساس مستند نصب خود ابزار عمل شده است.

```
root@skye ~/ukibak (master)#
root@skye ~/ukibak (master)# install -Dm0750 -t "/usr/sbin/" "target/release/ukibak"
root@skye ~/ukibak (master)# install -Dm0750 -t "/etc/systemd/system/" "systemd/ukibak.service"
root@skye ~/ukibak (master)# install -Dm0750 -t "/etc/systemd/system/" "systemd/ukibak.timer"
root@skye ~/ukibak (master)#
```

حالا می‌توان سرویس‌های مربوط به پشتیبانی گیری از UKI توسط ابزار ukibak را فعال نمود.

```
root@skye ~/ukibak (master)#
root@skye ~/ukibak (master)# systemctl start ukibak
root@skye ~/ukibak (master)# systemctl enable ukibak.timer
root@skye ~/ukibak (master)# systemctl start ukibak.timer
root@skye ~/ukibak (master)#
```

همانطور که مشاهده می‌شود با استفاده از ابزار ukibak برای اولین بار به صورت دستی، نسخه پشتیبانی UKI سیستم عامل در مسیر /boot/EFI/Linux/linux-last.efi ایجاد شده است.

```
root@skye ~#  
root@skye ~# ls /boot/EFI/Linux/  
arch.efi*  
root@skye ~# ukibak  
INFO [ukibak] Copying '/boot/EFI/Linux/arch.efi' to '/boot/EFI/Linux/linux-last.efi'...  
root@skye ~# ls /boot/EFI/Linux/  
arch.efi* linux-last.efi*  
root@skye ~# _
```

در ادامه می‌توان نسخه پشتیبانی گرفته شده UKI را با استفاده از دستور efibootmgr به عنوان یکی از اولویت‌های بوت اضافه نمود.

```
root@skye ~#  
root@skye ~# efibootmgr -c -d /dev/sda -p 1 -L "Last Good Arch UKI" -l "\EFI\Linux\linux-last.efi"  
BootCurrent: 0003  
Timeout: 0 seconds  
BootOrder: 0004,0003,0000,0001,0002  
Boot0000* UiApp FvVol(7cb8bdc9-f8eb-4f34-aaea-3ee4af6516a1)/FvFile(462caa21-7614-4503-836e-8ab6f4662331)  
Boot0001* UEFI VBOX CD-ROM VBO-01f003f6 PciRoot(0x0)/Pci(0x1,0x1)/Ata(0,0,0){auto_created_boot_option}  
Boot0002* UEFI VBOX HARDISK VBF5825ef9-9b05fa18 PciRoot(0x0)/Pci(0xd,0x0)/Sata(0,65535,0){auto_created_boot_opti  
Boot0003* Arch UKI HD(1,GPT,3a3c4c4b-89cc-4a52-8a3d-474dcef969be,0x800,0x100000)\EFI\Linux\arch.efi  
Boot0004* Last Good Arch UKI HD(1,GPT,3a3c4c4b-89cc-4a52-8a3d-474dcef969be,0x800,0x100000)\EFI\Linux\linux-last.efi  
root@skye ~#
```

تصویر زیر نسخه بوت شده سیستم عامل با استفاده از نسخه پشتیبانی UKI را نشان می‌دهد. برای اینکه سیستم عامل همچنان با استفاده از UKI اصلی بوت شود، اولویت‌های بوت تغییر داده شده و نسخه پشتیبانی گرفته شده در اولویت دوم قرار داده شده است.

```
root@skye ~#  
root@skye ~# efibootmgr --bootorder 0003,0004,0000,0001,0002  
BootCurrent: 0004  
Timeout: 0 seconds  
BootOrder: 0003,0004,0000,0001,0002  
Boot0000* UiApp FvVol(7cb8bdc9-f8eb-4f34-aaea-3ee4af6516a1)/FvFile(462caa21-7614-4503-836e-8ab6f4662331)  
Boot0001* UEFI VBOX CD-ROM VBO-01f003f6 PciRoot(0x0)/Pci(0x1,0x1)/Ata(0,0,0){auto_created_boot_option}  
Boot0002* UEFI VBOX HARDISK VBF5825ef9-9b05fa18 PciRoot(0x0)/Pci(0xd,0x0)/Sata(0,65535,0){auto_created_boot_opti  
Boot0003* Arch UKI HD(1,GPT,3a3c4c4b-89cc-4a52-8a3d-474dcef969be,0x800,0x100000)\EFI\Linux\arch.efi  
Boot0004* Last Good Arch UKI HD(1,GPT,3a3c4c4b-89cc-4a52-8a3d-474dcef969be,0x800,0x100000)\EFI\Linux\linux-last.efi  
root@skye ~#
```

بخش ۳.۴

برای نمایش یک نمونه حمله ساده Evil Maid، می‌توان یک hook جدید به initramfs اضافه نموده و یک UKI جدید با استفاده از این تغییرات ایجاد نمود.

بدین منظور، ابتدا نیاز است تا hook مورد نظر ایجاد شود. این hook در مسیر `/etc/initcpio/hooks/demo_evil_maid` قرار داده شده است. این hook یک فایل دخیره شده در رم در مسیر `/run/demo/event.log` می‌سازد و یک رشته مشخص را در آن قرار می‌دهد.

```
root@skye ~#  
root@skye ~# vim /etc/initcpio/hooks/demo_evil_maid
```

```
run_hook() {  
    mkdir -p /run/demo  
  
    echo "Hook has been ran." > /run/demo/event.log  
}
```

در ادامه این hook، در تنظیمات مربوط به mkinitcpio تعریف شده است.

```
root@skye ~#  
root@skye ~# vim /etc/mkinitcpio.conf
```

```
HOOKS=(base udev autodetect modconf block encrypt demo_evil_maid filesystems fsck)  
"/etc/mkinitcpio.conf" 84L, 3430B
```

حالا با استفاده از دستور mkinitcpio می‌توان initramfs را بار دیگر به منظور اعمال تغییرات ایجاد شده، تولید نمود.

```
root@skye ~#  
root@skye ~# mkinitcpio -P  
==> Building image from preset: /etc/mkinitcpio.d/linux.preset: 'default'  
==> Using default configuration file: '/etc/mkinitcpio.conf'  
-> -k /boot/vmlinuz-linux -g /boot/initramfs-linux.img  
==> Starting build: '7.0.10-arch1-1'  
-> Running build hook: [base]  
-> Running build hook: [udev]  
-> Running build hook: [autodetect]  
-> Running build hook: [modconf]  
-> Running build hook: [block]  
-> Running build hook: [encrypt]  
==> WARNING: Possibly missing firmware for module: 'qat_6xxx'  
-> Running build hook: [demo_evil_maid]  
-> Running build hook: [filesystems]  
-> Running build hook: [fsck]  
==> Generating module dependencies  
==> Creating zstd-compressed initcpio image: '/boot/initramfs-linux.img'  
-> Early uncompressed CPPIO image generation successful  
==> Initcpio image generation successful  
==> Running post hooks  
-> Running post hook: [sbctl]  
Secureboot key directory doesn't exist, not signing!  
==> Post processing done  
root@skye ~#
```

در ادامه، بار دیگر UKI ذخیره شده در مسیر `/boot/EFI/Linux/arch.efi` ایجاد شده با این تفاوت که این UKI توسط evil maid، با استفاده hook نوشته شده، آلوده شده است.

```
root@skye ~#  
root@skye ~# ukify build \  
--linux /boot/vmlinuz-linux \  
--initrd /boot/initramfs-linux.img \  
--initrd /boot/intel-ucode.img \  
--cmdline "root=UUID=$(blkid -s UUID -o value /dev/sda2) rw" \  
--output /boot/EFI/Linux/arch.efi  
Kernel version not specified, starting autodetection ■ .  
Found uname version: 7.0.10-arch1-1  
Wrote unsigned /boot/EFI/Linux/arch.efi  
root@skye ~#
```

با راه اندازی مجدد سیستم عامل، مشاهده می شود که hook مورد نظر در زمان بوت اجرا شده بوده و رشته مشخص در مسیر تعریف شده وجود قرار گرفته است.

```
root@skye ~#  
root@skye ~# cat /run/demo/event.log  
Hook has been ran.  
root@skye ~# _
```

برای بازیابی از این وضعیت، می توان UKI پشتیبانی گرفته شده توسط ابزار ukibak را به جای `/boot/EFI/Linux/arch.efi` قرار داد تا سیستم عامل به حالت اولیه خودش بازگردد.

```
root@skye /b/E/Linux#  
root@skye /b/E/Linux# ls -lh  
total 93M  
-rwxr-xr-x 1 root root 48M May 31 17:14 arch.efi*  
-rwxr-xr-x 1 root root 45M May 29 23:02 linux-last.efi*  
root@skye /b/E/Linux# cp linux-last.efi arch.efi  
root@skye /b/E/Linux# _
```

با راه اندازی مجدد سیستم عامل، مشاهده می شود که hook مورد نظر اینبار اجرا نشده و فایلی در مسیر تعریف شده وجود ندارد.

```
root@skye ~#  
root@skye ~# cat /run/demo/event.log  
cat: /run/demo/event.log: No such file or directory  
root@skye ~ [1]# _
```


ukibak در اصل یک ابزار بکاپ‌گیری، خودکار از UKI است که با هدف افزایش قابلیت بازیابی طراحی شده و نه ایجاد یک لایه امنیتی مستقل. این ابزار قبل از اینکه از UKI فعلی یک کپی به عنوان بکاپ بسازد چند بررسی انجام می‌دهد تا مطمئن شود چیزی که ذخیره می‌کند واقعا قابل استفاده و سالم است.

به طور مثال، اول بررسی می‌شود که سیستم در حال حاضر از روی همان بکاپ در حال اجرا نباشد، چون اگر این اتفاق بیفتد عملا بکاپ بی‌معنی می‌شود و فقط نسخه تکراری ساخته می‌شود. در مرحله بعد بررسی می‌کند که فایل مورد نظر واقعا یک UKI معتبر شامل اجزای اصلی مانند kernel، initrd و kernel command line باشد. این بررسی‌ها باعث می‌شود چیزی که به عنوان بکاپ ذخیره می‌شود یک ایمج ناقص یا اشتباه نباشد، چون اگر چنین چیزی بکاپ گرفته شود، در زمان recovery عملا سیستم غیرقابل بوت می‌شود. یک شرط مهم دیگر برای بکاپ‌گیری این است که image فعلی حداقل یکبار با موفقیت بوت شده و برای مدت زمان معینی stable باشد. این باعث می‌شود احتمال اینکه بکاپ خودش عامل خرابی شود کمتر شود.

از نظر نقاط قوت، این طراحی باعث می‌شود سیستم در برابر خرابی‌های ناشی از آپدیت، اشتباهات کاربر یا مشکل ساخت UKI مقاوم‌تر شود و همیشه یک نسخه قابل بوت داشته باشد. همچنین چون تصمیم‌ها بر اساس شرایط واقعی بوت گرفته می‌شوند، احتمال اینکه بکاپ خراب یا غیرقابل استفاده ساخته شود کم می‌شود.

اما محدودیت مهم این است که ukibak عملا یک ابزار امنیتی نیست بلکه فقط یک مکانیزم کمکی است. یعنی اگر سیستم از قبل compromise شده باشد یا اگر مشکل در سطح امنیتی مثل کلیدهای Secure Boot یا firmware وجود داشته باشد، این ابزار هیچ کمکی نمی‌کند. همچنین معیار بوت موفق برای این ابزار، یک معیار نسبی است و ممکن است سیستم بعد از بوت اولیه دچار مشکل شود ولی همچنان به عنوان سالم برای بکاپ در نظر گرفته شود.

از نظر مقاومت در برابر Evil Maid هم تاثیر آن محدود است. این ابزار جلوی دستکاری مستقیم یا حمله به زنجیره اعتماد را نمی‌گیرد، بلکه فقط باعث می‌شود اگر یک نسخه خراب یا غیرقابل بوت وارد سیستم شود، کاربر سریع‌تر بتواند به نسخه قبلی برگردد. بنابراین اثر اصلی آن روی availability و recovery است. اگر مهاجم بتواند در سطح EFI یا Secure Boot دستکاری انجام دهد، ukibak نمی‌تواند جلوی آن را بگیرد و فقط ممکن است در بهترین حالت فرآیند بازیابی را ساده‌تر کند. با توجه به نمونه‌ای که در بخش قبلی نیز اجرا شد این ابزار به دلیل موفق بوت شدن سیستم و کارکرد بدون مشکل آن، پس از مدتی از UKI آلوده شده بکاپ تهیه نموده و حمله Evil Maid را تشخیص نخواهد داد.